



Documentación del Trabajo Práctico: Chatbot Escolar con IA

1. Introducción y Contexto

El objetivo de este trabajo práctico es diseñar, programar e implementar un **asistente virtual (Chatbot)** capaz de resolver dudas recurrentes sobre trámites administrativos y académicos de nuestra escuela técnica. El sistema procesará el lenguaje natural de los usuarios mediante una **red neuronal en Python** y se expondrá como un servicio en red (API) para ser integrado en la página web de la institución.

2. Requerimientos Funcionales (RF)

Los alumnos deberán garantizar que el sistema cumpla estrictamente con las siguientes funciones:

- **RF-01: Procesamiento de Lenguaje Natural (PLN):** El bot debe entender la intención del usuario final, incluso si introduce errores ortográficos, abreviaturas o sinónimos (ej: "constancia de alumno", "certificado regular", "papel para la obra social" deben activar la misma respuesta).
 - **RF-02: Gestión de Trámites Técnicos:** El sistema debe responder de forma precisa, como mínimo, a los siguientes ejes temáticos:
 - Requisitos y fechas para Prácticas Profesionalizantes / Pasantías.
 - Protocolo de seguridad y uso de herramientas en Talleres y Laboratorios.
 - Mesas de examen (previas y equivalencias).
 - Emisión de documentación (constancia de alumno regular, analíticos).
 - **RF-03: Interfaz de Programación de Aplicaciones (API REST):** El backend en Python debe exponer un *endpoint* HTTP (por ejemplo, `/chat` mediante método `POST`) que reciba la pregunta del usuario en formato JSON y devuelva la respuesta procesada por la IA.
 - **RF-04: Respuesta por Defecto (Fallback):** Si la red neuronal no logra superar un umbral mínimo de certeza (ej. menos del 60% de coincidencia), el bot debe responder con un mensaje genérico amable y derivar a un correo institucional o preceptoría.
-

3. Requerimientos No Funcionales (RNF)

- **RNF-01: Lenguaje de Programación:** El backend completo debe desarrollarse en **Python 3.x**.
 - **RNF-02: Tiempo de Respuesta:** El procesamiento de la red neuronal y la respuesta de la API no deben superar los **2 segundos** por consulta en condiciones normales de red.
 - **RNF-03: Arquitectura desacoplada:** La lógica del chatbot debe estar completamente separada de la interfaz gráfica (Frontend web).
-

4. Arquitectura Tecnológica Sugerida (Para los Alumnos)

Para mantener el proyecto didáctico, robusto y sin costos de APIs externas, se sugiere a los alumnos la siguiente pila tecnológica:

[Frontend: Web de la Escuela]

| (Fetch API / JSON)



[Backend en Red: Flask o FastAPI]

| (Controlador de rutas)



[Procesamiento: Red Neuronal con Keras/TensorFlow o ChatterBot]

1. **Frontend (Página de la escuela):** Un archivo HTML/CSS/JS básico con una ventana de chat flotante que use `fetch()` para enviar datos al servidor Python.
 2. **Servidor de Red (Backend): Flask o FastAPI.** Son frameworks ligeros en Python ideales para enseñar cómo levantar un servidor HTTP y manipular rutas en la materia de redes.
 3. **Capa de IA (Red Neuronal):**
 - *Opción Académica Estándar:* Utilizar **TensorFlow/Keras** para crear una red neuronal secuencial densa (MLP) que clasifique intenciones a partir de un archivo de entrenamiento `intents.json`.
 - *Opción Simplificada:* Utilizar librerías de procesamiento de texto combinadas como `scikit-learn` (utilizando TF-IDF y clasificadores) o herramientas NLP específicas nativas de Python.
-

5. Estructura de Datos de Entrenamiento (`intents.json`)

Los alumnos deberán diseñar un archivo de configuración que sirva de base de conocimiento para alimentar la red neuronal. Ejemplo de estructura:

```
{
  "intents": [
    {
      "tag": "alumno_regular",
      "patterns": [
        "¿Cómo saco la constancia de alumno regular?",
        "Necesito un certificado de estudio",
        "Papel para la obra social",
        "Constancia de alumno"
      ],
      "responses": [
        "Puedes solicitar tu constancia de alumno regular en Secretaría de lunes a viernes de 8:00 a 12:00 hs, o pedirla de forma digital enviando un correo a secretaria@escuelatecnica.edu.ar con tus datos."
      ]
    },
    {
      "tag": "pasantias",
      "patterns": [
        "¿Cuándo empiezan las prácticas profesionalizantes?",
        "Requisitos para las pasantías",
        "¿Qué necesito para hacer las prácticas?"
      ],
      "responses": [
        "Las Prácticas Profesionalizantes inician formalmente en el segundo cuatrimestre para alumnos de 6.º año. Debes tener los talleres del ciclo superior aprobados y presentar la ficha médica actualizada al Jefe de Área."
      ]
    }
  ]
}
```

6. Pautas de Evaluación y Entregables

Para aprobar el trabajo práctico, los grupos de alumnos deberán presentar:

1. **Código Fuente:** Repositorio en GitHub/GitLab con el código ordenado y un archivo `README.md` que explique cómo instalar las dependencias (`pip install -r requirements.txt`) y ejecutar el servidor local.
 2. **Script de Entrenamiento:** El archivo Python independiente que toma el `intents.json`, preprocesa el texto (tokenización, lematización), entrena la red neuronal y guarda el modelo (ej. `chatbot_model.h5`).
 3. **Demostración de Red (Testeo de API):** Evidencia de que el servidor responde correctamente realizando peticiones mediante herramientas de red como **Postman**, **Insomnia** o comandos `curl`.
-

Ejemplo (Código Base)

1. Archivo de Entrenamiento (`train.py`)

Este script se encarga de leer el archivo `intents.json`, preprocesar el texto utilizando técnicas básicas de bolsa de palabras (*Bag of Words*), entrenar una **red neuronal secuencial (MLP)** y guardar el modelo entrenado.

```
import json
import numpy as np
import pickle
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# 1. Cargar la base de conocimiento
with open('intents.json', 'r', encoding='utf-8') as f:
    intents = json.load(f)

words = []
classes = []
documents = []
ignore_letters = ['?', '¿', '!', '¡', '.', ',', ';', ':']

# 2. Preprocesamiento básico (Tokenización manual por espacios)
for intent in intents['intents']:
    for pattern in intent['patterns']:
        # Tokenizar palabras de forma simple y pasarlas a minúsculas
        word_list = [w.lower().strip('?!.,¿¡') for w in pattern.split()]
        words.extend(word_list)
        documents.append((word_list, intent['tag']))
        if intent['tag'] not in classes:
            classes.append(intent['tag'])

# Limpiar duplicados y ordenar
words = sorted(list(set([w for w in words if w not in ignore_letters])))
classes = sorted(list(set(classes)))

# Guardar estructuras para usar en el servidor
pickle.dump(words, open('words.pkl', 'wb'))
pickle.dump(classes, open('classes.pkl', 'wb'))

# 3. Crear el set de entrenamiento (Bag of Words)
training = []
output_empty = [0] * len(classes)

for doc in documents:
```

```

bag = []
word_patterns = doc[0]
for word in words:
    bag.append(1) if word in word_patterns else bag.append(0)

output_row = list(output_empty)
output_row[classes.index(doc[1])] = 1
training.append([bag, output_row])

# Mezclar y convertir a arreglos de NumPy
np.random.shuffle(training)
training = np.array(training, dtype=object)

train_x = np.array(list(training[:, 0]))
train_y = np.array(list(training[:, 1]))

# 4. Arquitectura de la Red Neuronal (Modelado Keras)
model = Sequential([
    Dense(128, input_shape=(len(train_x[0]),), activation='relu'),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(len(classes), activation='softmax') # Clasificación multi-clase
])

# Compilación y Entrenamiento
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.fit(train_x, train_y, epochs=200, batch_size=5, verbose=1)

# Guardar el modelo entrenado
model.save('chatbot_model.h5')
print("¡Modelo entrenado y guardado con éxito!")

```

2. Archivo del Servidor de Red (`app.py`)

Este archivo levanta un servidor de red local usando **Flask**. Expone el endpoint `/chat` que recibirá las preguntas desde el frontend y utilizará la red neuronal guardada para responder en formato JSON.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import json
import numpy as np
import pickle
import tensorflow as tf

app = Flask(__name__)
CORS(app) # Permite solicitudes desde el frontend de la escuela

# Cargar el modelo de IA y las estructuras de datos
model = tf.keras.models.load_model('chatbot_model.h5')
words = pickle.load(open('words.pkl', 'rb'))
classes = pickle.load(open('classes.pkl', 'rb'))

with open('intents.json', 'r', encoding='utf-8') as f:
    intents = json.load(f)

# Función auxiliar para convertir la entrada en Bag of Words
def clean_up_sentence(sentence):
    return [w.lower().strip('?!.,;') for w in sentence.split()]

def bow(sentence, words):
    sentence_words = clean_up_sentence(sentence)
    bag = [0] * len(words)
    for s in sentence_words:
        for i, word in enumerate(words):
            if word == s:
                bag[i] = 1
    return np.array(bag)

# RUTA DE RED PRINCIPAL (API REST)
@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json()
    user_message = data.get("message", "")

    if not user_message:
        return jsonify({"response": "No enviaste ningún mensaje."}), 400

    # 1. Predecir con la red neuronal
```

```
p = bow(user_message, words)
res = model.predict(np.array([p]))[0]

# 2. Filtrar por umbral de certeza (Requerimiento No Funcional)
ERROR_THRESHOLD = 0.60
results = [[i, r] for i, r in enumerate(res) if r > ERROR_THRESHOLD]
results.sort(key=lambda x: x[1], reverse=True)

# 3. Determinar respuesta o aplicar Fallback
if results:
    tag = classes[results[0][0]]
    for i in intents['intents']:
        if i['tag'] == tag:
            response_text = i['responses'][0]
            break
else:
    # Mensaje por defecto si la red neuronal no está segura
    response_text = "Lo siento, no logré entender tu consulta sobre el trámite. Por favor,
comunícate con Secretaría al correo secretaria@escuelatecnica.edu.ar."

return jsonify({"response": response_text})

if __name__ == '__main__':
    # El servidor corre en el puerto 5000 por defecto
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Este archivo simula la ventana de chat flotante y se conecta directamente por red con el servidor de Python que corre en el puerto 5000 (<http://localhost:5000/chat>).

Interfaz de la Ventana de Chat ([index.html](#))

Podés entregarles este archivo base. Es simple, funcional y utiliza una estructura limpia para que los alumnos se concentren en cómo viajan los datos en formato JSON a través del protocolo HTTP.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Chatbot Escuela Técnica</title>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f4f4f9; display: flex;
justify-content: center; align-items: center; height: 100vh; margin: 0; }
    .chat-container { width: 400px; height: 500px; background: white; border-radius: 10px;
box-shadow: 0 4px 15px rgba(0,0,0,0.1); display: flex; flex-direction: column; overflow:
hidden; }
    .chat-header { background-color: #0056b3; color: white; padding: 15px; text-align:
center; font-weight: bold; }
    .chat-box { flex: 1; padding: 15px; overflow-y: auto; background-color: #fafafa; display:
flex; flex-direction: column; gap: 10px; }
    .message { padding: 10px 14px; border-radius: 15px; max-width: 75%; font-size: 14px;
line-height: 1.4; }
    .user-message { background-color: #007bff; color: white; align-self: flex-end;
border-bottom-right-radius: 0; }
    .bot-message { background-color: #e9ecef; color: #333; align-self: flex-start;
border-bottom-left-radius: 0; }
    .chat-input-area { display: flex; border-top: 1px solid #ddd; padding: 10px; background:
#fff; }
    .chat-input-area input { flex: 1; padding: 10px; border: 1px solid #ccc; border-radius:
5px; outline: none; }
    .chat-input-area button { background-color: #0056b3; color: white; border: none;
padding: 0 15px; margin-left: 5px; border-radius: 5px; cursor: pointer; }
    .chat-input-area button:hover { background-color: #004085; }
  </style>
</head>
<body>

<div class="chat-container">
  <div class="chat-header">🤖 Asistente de Trámites Escolares</div>
  <div class="chat-box" id="chatBox">
    <div class="message bot-message">¡Hola! Soy el asistente virtual. ¿Qué duda tenés
sobre los trámites, talleres o pasantías de la escuela?</div>
  </div>
  <div class="chat-input-area">
```

```

    <input type="text" id="userInput" placeholder="Escribí tu consulta acá..."
onkeypress="handleKeyPress(event)">
    <button onclick="sendMessage()">Enviar</button>
</div>
</div>

<script>
// Dirección de red de la API de Python (servidor Flask)
const API_URL = 'http://localhost:5000/chat';

// Función principal para enviar el mensaje por la red
async function sendMessage() {
    const inputField = document.getElementById('userInput');
    const chatBox = document.getElementById('chatBox');
    const text = inputField.value.trim();

    if (text === "") return;

    // 1. Mostrar el mensaje del usuario en la pantalla
    appendMessage(text, 'user-message');
    inputField.value = "";

    try {
        // 2. Realizar la petición de red (HTTP POST) usando fetch()
        const response = await fetch(API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ message: text }) // Convertimos el objeto en texto JSON
        });

        if (!response.ok) {
            throw new Error('Error en la respuesta del servidor');
        }

        // 3. Procesar la respuesta JSON que devuelve la Red Neuronal
        const data = await response.json();

        // 4. Mostrar la respuesta del bot en pantalla
        appendMessage(data.response, 'bot-message');

    } catch (error) {
        console.error('Error de red:', error);
        appendMessage('Error de red: No se pudo conectar con el servidor del chatbot.',
'bot-message');
    }
}

```

```
// Función auxiliar para agregar las burbujas de texto al contenedor visual
function appendMessage(text, className) {
  const chatBox = document.getElementById('chatBox');
  const messageDiv = document.createElement('div');
  messageDiv.classList.add('message', className);
  messageDiv.innerText = text;
  chatBox.appendChild(messageDiv);

  // Auto-scroll hacia el último mensaje recibido
  chatBox.scrollTop = chatBox.scrollHeight;
}

// Permite mandar el mensaje al presionar la tecla 'Enter'
function handleKeyPress(event) {
  if (event.key === 'Enter') {
    sendMessage();
  }
}
</script>

</body>
</html>
```



Rúbrica de Evaluación

Criterio de Evaluación	Destacado (4 pts)	Regular / En Proceso (2 pts)	Insuficiente (0 pts)
Configuración de Red e Integración (API REST)	El backend expone correctamente el endpoint mediante Flask/FastAPI, maneja CORS y responde solicitudes JSON en menos de 2 seg de forma desacoplada.	El servidor funciona pero tiene problemas de conectividad con el Frontend, o mezcla la interfaz web dentro del código Python de forma acoplada.	El servidor no levanta, no recibe peticiones por red o falla al procesar el protocolo HTTP de forma general.
Lógica de la Red Neuronal y Entrenamiento	El modelo entrena sin errores, se guarda en disco y clasifica las intenciones escolares usando funciones de activación y pérdida adecuadas.	La red neuronal compila pero el archivo de conocimiento (<code>intents.json</code>) está incompleto o tiene muy pocos patrones para generalizar.	No se evidencia el uso de redes neuronales o el script de entrenamiento genera errores críticos de NumPy/TensorFlow que impiden compilar.
Manejo de Errores y Fallback	El sistema detecta cuando una pregunta no pertenece a ningún trámite (aplica correctamente el filtro de porcentaje de confianza) y da una respuesta genérica.	El chatbot intenta responder a todo, arrojando respuestas incoherentes o incorrectas frente a preguntas sin sentido (falla el umbral de certeza).	El backend se cae por completo (<i>crash</i>) si el usuario introduce caracteres extraños o realiza una consulta imprevista.
Buenas Prácticas de Desarrollo y Entrega	Código modular bien comentado, uso de un entorno virtual (<code>venv</code>), archivo <code>requirements.txt</code> completo y repositorio Git prolijo con historial de commits.	El código funciona pero está todo en un único archivo desorganizado, sin comentarios y no incluye instrucciones claras de instalación.	Entrega de archivos sueltos por canales informales, código copiado directamente sin modificaciones o dependencias rotas.